

# Common Part 2: Complexity Classes, Turing Machines, Randomization, and Undecidability

## Common Part 2: TAL

Problem/language complexity classes with respect to the time complexity of their solution and memory complexity including undecidable problems/languages.

**Primary source repositories consulted:** mohwald/oi-mszz, pan-sveta/oi-si-statnice, Pryx/oi-si-mszz, dralii/oi-mszz. The final text below is synthesized and corrected against standard theory of computation.

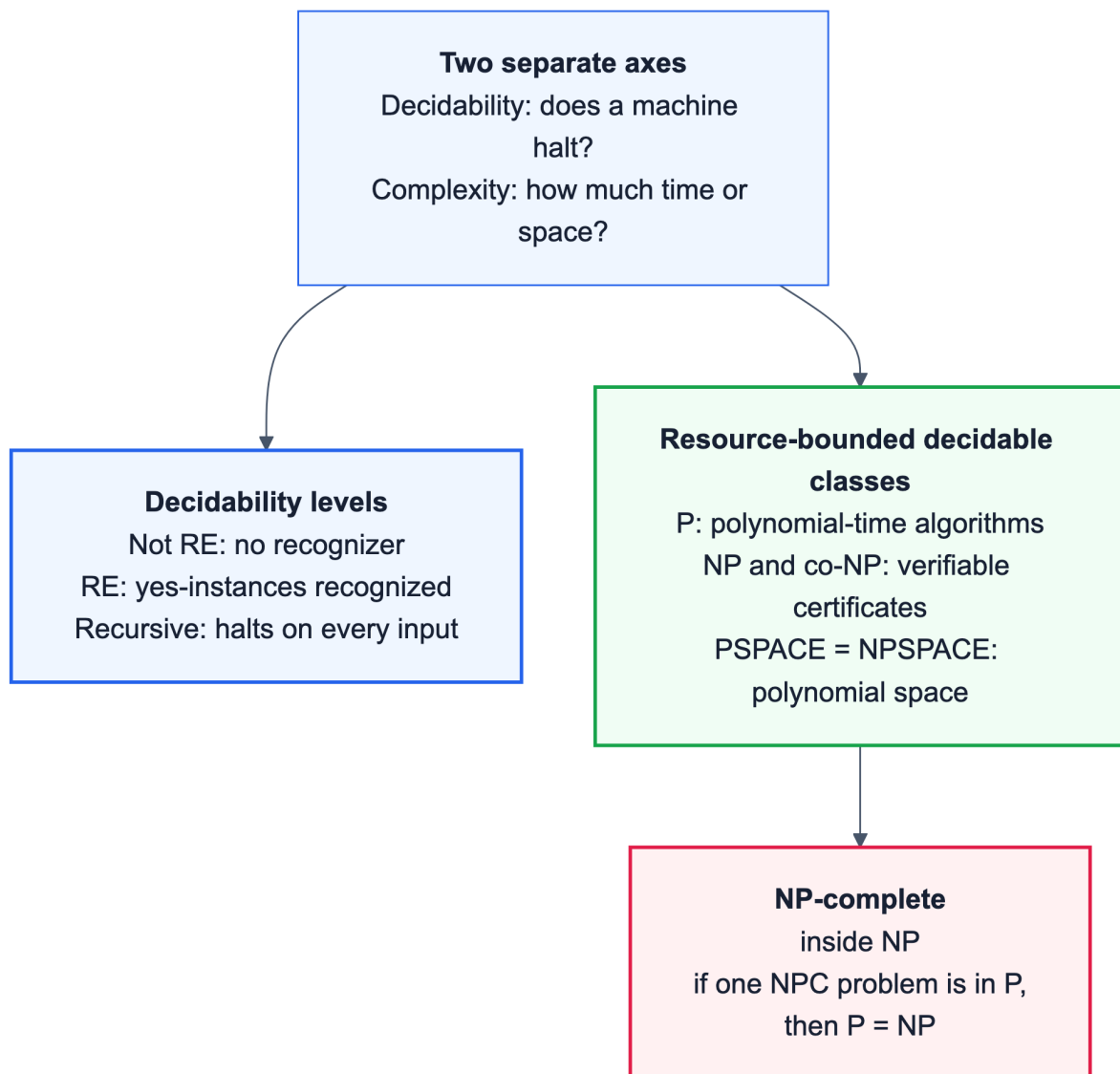


Figure 1: TAL hierarchy: resource-bounded classes sit inside decidable languages, while decidability classes form a separate axis.

# TAL in Plain English

TAL is about sorting problems by what machines can solve, how fast they can solve them, how much memory they need, and which problems cannot be solved at all.

## 1. Identify the world.

Time, space, randomness, or decidability?

## 2. Name the class.

P, NP, PSPACE, RP, RE, decidable...

## 3. Give the reason.

Algorithm, certificate, reduction, or contradiction.

### Complexity basics

What resources does an algorithm use?

- Time = number of steps.
- Space = working memory, usually not counting read-only input.
- Correctness = invariant for truth, variant for termination.

### Turing machines

The formal model behind algorithms.

- Deterministic: one next move.
- Nondeterministic: many branches; accept if one branch accepts.
- Multitape changes speed polynomially, not the big classes.

### P, NP, reductions

Can we solve it fast, or verify a solution fast?

- **P**: polynomial-time solving.
- **NP**: polynomial-time verification/certificate.
- **NP-hard**: everything in NP reduces to it.

### Space and randomness

Memory-bounded and probability-bounded computation.

- **PSPACE**: polynomial memory.
- Savitch:  $\text{NPSpace} = \text{PSPACE}$ .
- RP/co-RP/ZPP differ by one-sided or zero error.

### Undecidability

Some yes/no questions have no always-correct terminating algorithm.

- Decidable: halts with yes/no on every input.
- RE: halts on yes instances, may loop on no.
- Halting problem: classic contradiction by diagonalization.

### How to answer

Most TAL answers are classification plus proof idea.

- Define the class/problem.
- State the inclusion or theorem.
- Use reduction, certificate, simulation, or diagonalization.

**Exam sentence:** NP means solutions can be verified in polynomial time; NP-complete problems are in NP and at least as hard as every NP problem by polynomial reduction.

### Three traps

- NP does not mean "not polynomial"; P may still equal NP.
- Reduce from a known hard problem to prove your target hard.
- Semi-decidable is not the same as decidable.

Use this page first; then drill the formal definitions and theorem statements.

## 1. Asymptotic Growth, Complexity, and Correctness

### Asymptotic growth

For non-negative functions  $f, g$ :

- $f \in O(g)$ :  $f$  is eventually bounded above by a constant multiple of  $g$ .
- $f \in \Omega(g)$ :  $f$  is eventually bounded below by a constant multiple of  $g$ .
- $f \in \Theta(g)$ : both  $O(g)$  and  $\Omega(g)$ .
- $f \in o(g)$ :  $f/g \rightarrow 0$ .
- $f \in \omega(g)$ :  $f/g \rightarrow \infty$ .

Asymptotic notation ignores constants and lower-order terms. It describes growth as input size  $n$  tends to infinity.

### Time and space complexity

The **time complexity** of an algorithm is the number of elementary steps as a function of input size. The **space complexity** is the amount of working memory used as a function of input size.

For Turing machines:

- Time  $T(n)$ : maximum number of transitions on inputs of length  $n$ .
- Space  $S(n)$ : maximum number of work-tape cells used on inputs of length  $n$ .

Usually the read-only input tape is not counted toward space complexity.

Complexity of a **problem** is the complexity of the best possible algorithm in the chosen computational model. For robust classes such as P, NP, PSPACE, the exact reasonable model does not matter up to polynomial factors.

### Correctness of algorithms

An algorithm is **partially correct** if, whenever it terminates, its output satisfies the specification. It is **totally correct** if it is partially correct and always terminates.

**Loop invariant** A loop invariant is a statement true:

1. before the loop starts,
2. before and after every iteration,
3. and strong enough that, together with loop termination, implies the postcondition.

Proof pattern:

- Initialization: invariant holds before first iteration.
- Maintenance: if it holds before an iteration, it holds after it.
- Termination: invariant plus negated loop guard implies correctness.

**Loop variant** A loop variant is a value from a well-founded ordered set, usually non-negative integers, that strictly decreases on every iteration. It proves termination because there is no infinite decreasing sequence.

## 2. Turing Machines

### Deterministic Turing machine

A deterministic Turing machine (DTM) is commonly defined as

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej}),$$

where:

- $Q$  is a finite set of states.
- $\Sigma$  is the input alphabet.
- $\Gamma \supseteq \Sigma$  is the tape alphabet, including blank symbol.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$  is a partial transition function.
- $q_0$  is the start state.
- $q_{acc}$  and  $q_{rej}$  are halting accept/reject states.

At each step the machine reads one tape cell, writes a symbol, moves the head, and changes state.

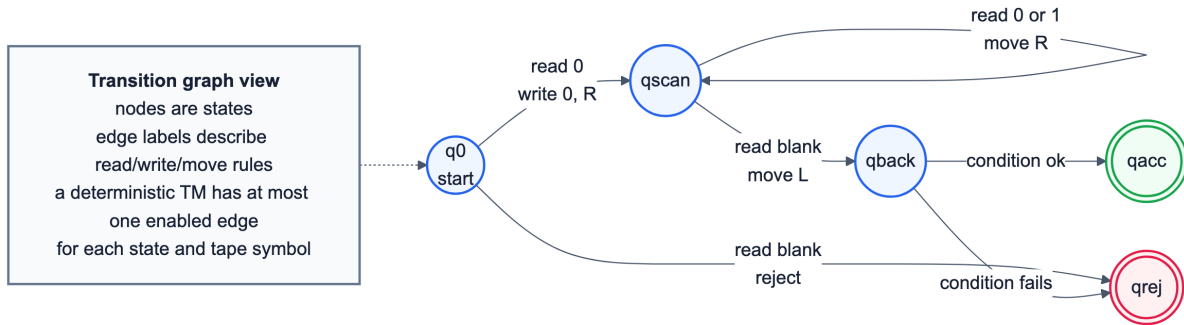


Figure 2: Turing-machine transition graph: states are nodes and transition rules are labeled directed edges.

A DTM **decides** a language if it halts on every input and accepts exactly strings in the language. It **recognizes** a language if it accepts exactly strings in the language, but may loop forever on strings outside the language.

### Multitape Turing machine

A multitape TM has several tapes and heads. In one step it reads all head symbols, writes all head symbols, moves each head, and changes state.

Multitape machines are not more powerful than single-tape machines. A single-tape machine can simulate a  $k$ -tape machine by encoding all tapes on one tape. If the multitape machine runs in  $T(n)$  time, a standard single-tape simulation runs in  $O(T(n)^2)$  time. Thus the difference does not affect polynomial-time classes.

### Nondeterministic Turing machine

A nondeterministic Turing machine (NTM) may have several possible transitions from one configuration. It accepts an input if at least one computation branch accepts.

Equivalent views:

- Branching computation tree.

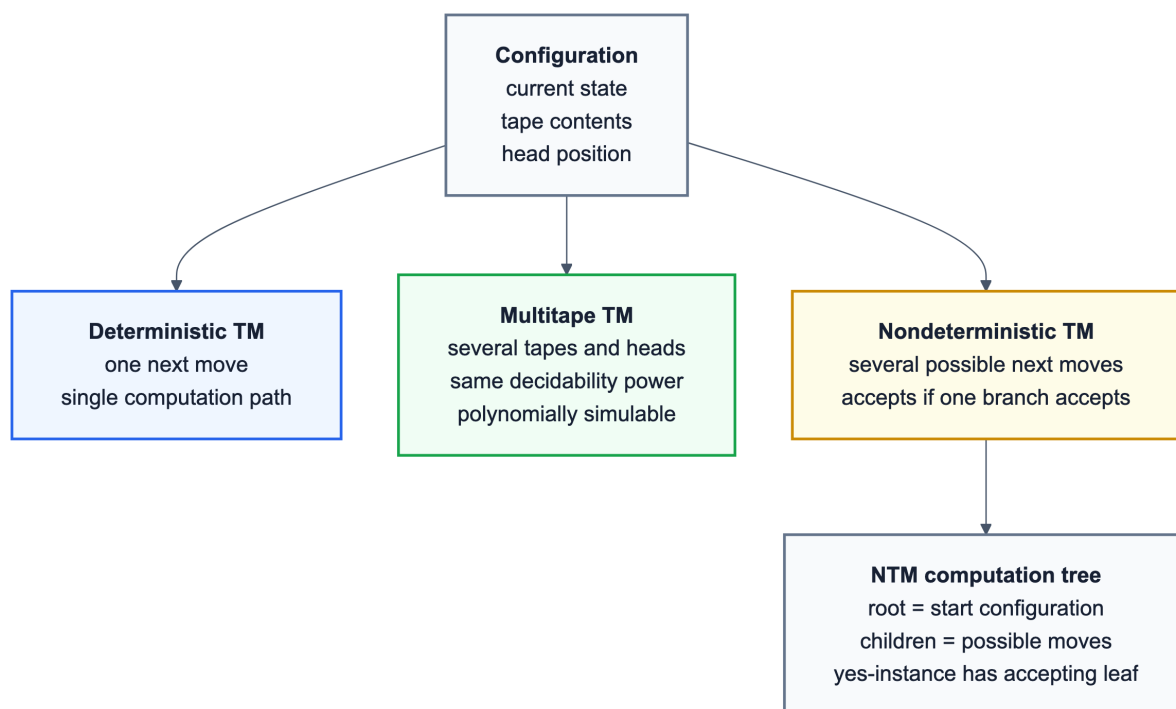


Figure 3: Turing-machine models: configurations lead to deterministic, multitape, and nondeterministic machine views.

- “Guess” a certificate and verify it deterministically.
- Existential computation: accept iff there exists an accepting branch.

For time complexity, an NTM runs in time  $T(n)$  if every branch halts within  $T(n)$  steps.

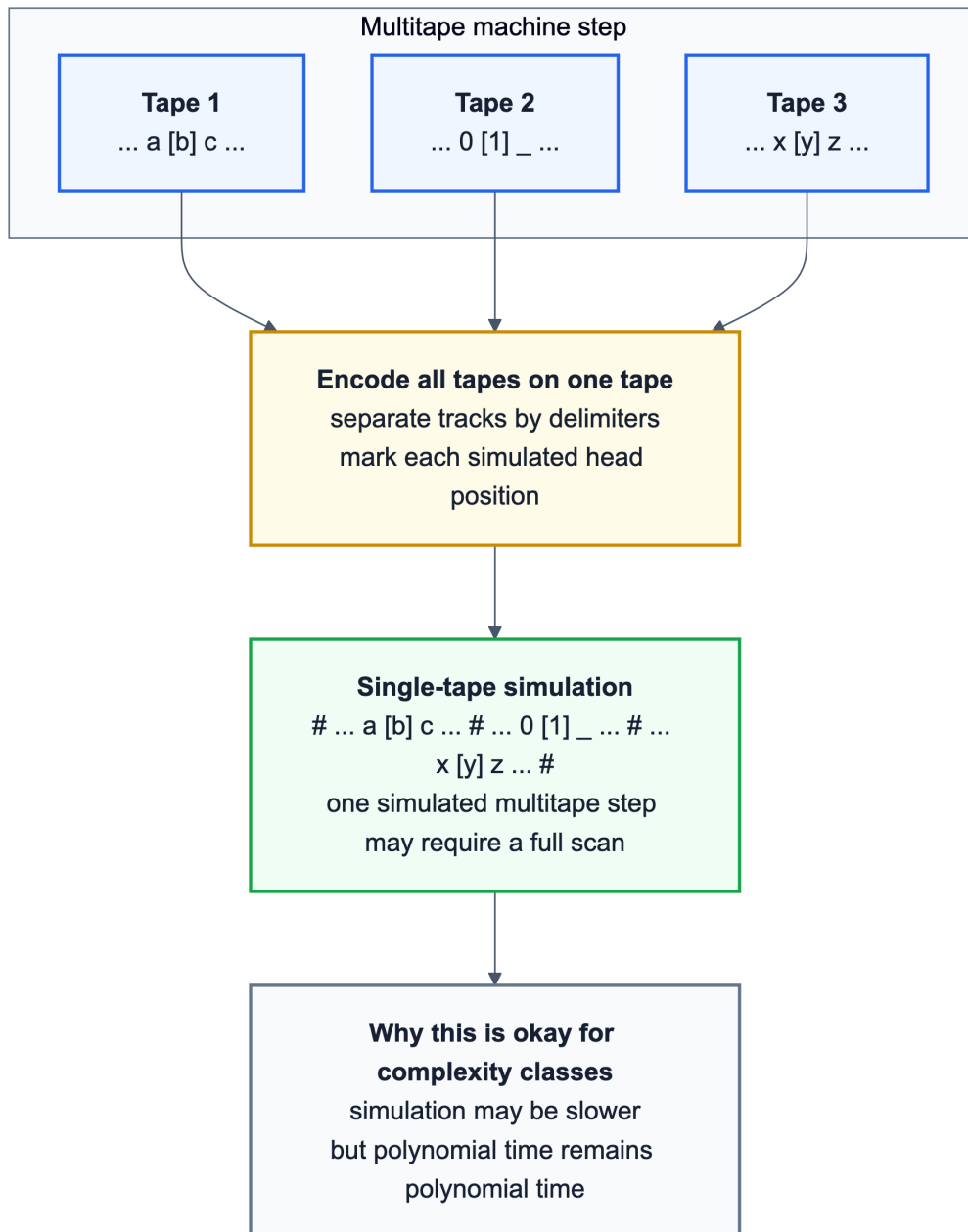


Figure 4: Multitape simulation: several tapes can be encoded on one tape using delimiters and marked head positions.

### 3. Decision Problems, Languages, P, NP, co-NP, Reductions, NPC

#### Decision problems and languages

A decision problem has yes/no answers. Once instances are encoded as strings over alphabet  $\Sigma$ , a decision problem corresponds to a language

$$L \subseteq \Sigma^*,$$

where  $x \in L$  means instance  $x$  is a yes-instance.

Optimization problems are often studied via decision versions. Example:

- Optimization TSP: find shortest tour.
- Decision TSP: is there a tour of length at most  $K$ ?

#### Class P

$$\mathbf{P} = \bigcup_{k \geq 1} \text{TIME}(n^k).$$

P is the class of languages decidable by a deterministic TM in polynomial time. It is the usual theoretical notion of efficiently solvable decision problems.

Examples: graph reachability, shortest path with nonnegative weights, MST decision version, bipartite matching, linear programming.

#### Class NP

$$\mathbf{NP} = \bigcup_{k \geq 1} \text{NTIME}(n^k).$$

Equivalently,  $L \in \mathbf{NP}$  iff there exists a polynomial-time deterministic verifier  $V$  and polynomial  $p$  such that

$$x \in L \iff \exists c, |c| \leq p(|x|) \text{ and } V(x, c) = 1.$$

The string  $c$  is a certificate/witness.

Examples: SAT, 3-SAT, CLIQUE, VERTEX-COVER, HAMILTONIAN-CYCLE, TSP decision version.

Clearly  $\mathbf{P} \subseteq \mathbf{NP}$ . Whether  $\mathbf{P} = \mathbf{NP}$  is open.

#### Simple explanation

P means “we can find the answer efficiently.” NP means “if someone gives us a proposed yes-answer certificate, we can check it efficiently.” Sudoku is a good intuition: solving may be hard, but checking a filled grid is easy.



## Class co-NP

For language  $L$ , its complement is  $\bar{L} = \Sigma^* \setminus L$ .

$$\mathbf{coNP} = \{L \mid \bar{L} \in \mathbf{NP}\}.$$

Problems in co-NP have efficiently verifiable no-certificates. Example: TAUT is in co-NP because its complement is SAT-like falsifiability.

We know  $\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$ . It is unknown whether  $\mathbf{NP} = \mathbf{coNP}$ .

## Polynomial many-one reductions

For languages  $A, B$ , a polynomial-time many-one reduction  $A \leq_p B$  is a polynomial-time computable function  $f$  such that

$$x \in A \iff f(x) \in B.$$

Interpretation: if we can solve  $B$  efficiently, then we can solve  $A$  efficiently by transforming instances.

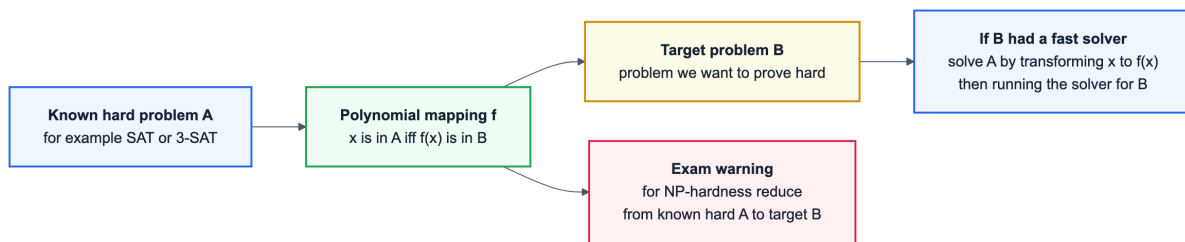


Figure 5: Polynomial reduction direction: reduce from a known hard problem to the target problem.

Use reductions to prove hardness:

1. Start from a known hard problem  $A$ .
2. Construct  $f$  mapping instances of  $A$  to instances of  $B$ .
3. Prove yes-instances map to yes-instances and no-instances to no-instances.
4. Conclude  $B$  is at least as hard as  $A$ .

### Example

To prove CLIQUE reduces to INDEPENDENT-SET, map graph  $G$  to its complement  $\bar{G}$  and keep the same  $k$ . A  $k$ -clique in  $G$  is exactly a  $k$ -independent set in  $\bar{G}$ , because edges become non-edges and non-edges become edges.

## NP-hard and NP-complete

A language  $B$  is **NP-hard** if every  $A \in \mathbf{NP}$  reduces to  $B$ :  $A \leq_p B$ .

A language  $B$  is **NP-complete** if:

1.  $B \in \mathbf{NP}$ ,
2.  $B$  is NP-hard.

If any NP-complete problem is in P, then  $\mathbf{P} = \mathbf{NP}$ .

### Exam tip

To prove a problem is NP-complete, show both parts: it is in NP, and it is NP-hard. For hardness, reduce from a known NP-complete problem to your problem, not the other way around.

## Cook-Levin theorem

Cook-Levin theorem: SAT is NP-complete.

Proof idea:

1. SAT is in NP: a truth assignment is a certificate, and formula evaluation is polynomial.
2. For any  $L \in \mathbf{NP}$ , take a polynomial-time NTM  $M$  deciding  $L$ .
3. Encode an accepting computation tableau of  $M$  on input  $x$  as a Boolean formula.
4. The formula is satisfiable iff  $M$  has an accepting computation branch on  $x$ .

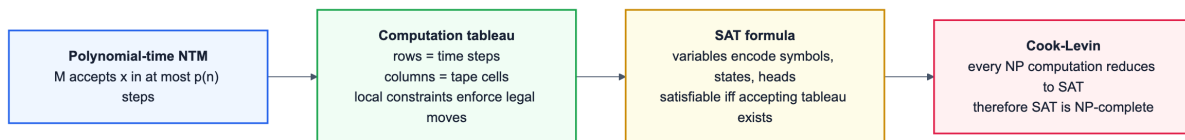


Figure 6: Cook-Levin tableau: a polynomial-time nondeterministic computation is encoded as a SAT formula.

This provides the first NP-complete problem. Many other NP-completeness proofs reduce from SAT or 3-SAT.

## Heuristics and approximation for NP-complete problems

For NP-hard optimization problems, exact polynomial-time algorithms are unlikely. Practical approaches:

- **Branch and bound:** exact, but exponential worst case; uses bounds to prune search.
- **Local search:** iteratively improve a candidate solution, e.g. k-OPT for TSP.
- **Greedy heuristics:** fast, sometimes with guarantees.
- **Randomized/metaheuristics:** simulated annealing, genetic algorithms, tabu search.
- **Approximation algorithms:** polynomial-time algorithms with provable ratio.

For minimization, algorithm  $A$  has approximation ratio  $\rho \geq 1$  if

$$A(I) \leq \rho \cdot \text{OPT}(I)$$

for every instance  $I$ . For maximization, use  $A(I) \geq \text{OPT}(I)/\rho$ .

Some problems have approximation schemes:

- PTAS: for every fixed  $\epsilon > 0$ , gives  $(1 + \epsilon)$ -approximation in polynomial time.
- FPTAS: polynomial in both input size and  $1/\epsilon$ .

**Example**

For metric TSP, a 2-approximation means the produced tour is never more than twice the optimal tour length. If  $OPT = 100$ , the algorithm promises a tour of length at most 200, although it may do better on a particular instance.

## 4. Space Complexity, PSPACE, NPSPACE, Savitch Theorem

### Space classes

For a function  $s(n)$ :

- $\text{DSPACE}(s(n))$ : languages decidable by a deterministic TM using  $O(s(n))$  space.
- $\text{NSPACE}(s(n))$ : languages decidable by a nondeterministic TM using  $O(s(n))$  space.

Polynomial space:

$$\mathbf{PSPACE} = \bigcup_{k \geq 1} \text{DSPACE}(n^k),$$

$$\mathbf{NPSPACE} = \bigcup_{k \geq 1} \text{NSPACE}(n^k).$$

Basic inclusions:

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE}.$$

The inclusion  $\mathbf{NP} \subseteq \mathbf{PSPACE}$  holds because a polynomial-time nondeterministic computation tree can be searched depth-first using only polynomial space.

### Configuration graph view

For a machine using  $s(n)$  space, a configuration contains state, head positions, and tape contents. The number of configurations is at most exponential in  $s(n)$ :

$$N = 2^{O(s(n))}.$$

Acceptance becomes reachability in this configuration graph.

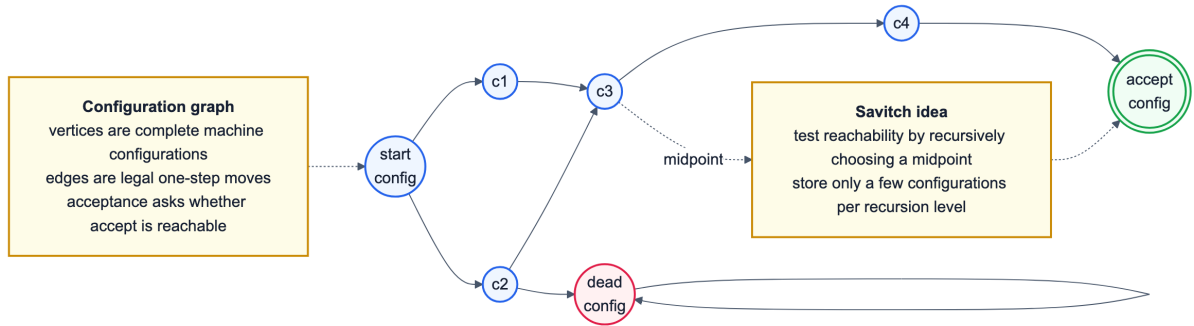


Figure 7: Configuration graph: accepting a space-bounded computation becomes a graph reachability question.

### Savitch theorem

For  $s(n) \geq \log n$ :

$$\text{NSPACE}(s(n)) \subseteq \text{DSPACE}(s(n)^2).$$

Therefore:

$$\mathbf{PSPACE} = \mathbf{NPSPACE}.$$

Proof idea: decide reachability between configurations  $u$  and  $v$  within at most  $t$  steps by recursively guessing a midpoint  $m$ :

$$\text{Reach}(u, v, t) = \exists m : \text{Reach}(u, m, \lceil t/2 \rceil) \wedge \text{Reach}(m, v, \lfloor t/2 \rfloor).$$

The recursion depth is  $O(\log t)$ , and each level stores a configuration of size  $O(s(n))$ . Since  $t$  can be exponential in  $s(n)$ ,  $\log t = O(s(n))$ , yielding  $O(s(n)^2)$  deterministic space.

**Simple explanation**

Savitch's theorem says nondeterminism saves much less space than it seems. Instead of storing the whole nondeterministic search tree, a deterministic machine recursively asks: "is there some middle configuration that connects the start to the goal?"

## 5. Randomized Algorithms and Randomized Complexity Classes

### Randomized algorithms

A randomized algorithm has access to random bits. Its output or running time is a random variable.

Two important types:

- **Monte Carlo:** bounded running time, may return wrong answer with small probability.
- **Las Vegas:** always correct, running time is random; expected running time is bounded.

Randomization is useful for symmetry breaking, hashing, sampling, primality testing, randomized quicksort, and randomized reductions.

### Randomized Turing machine

A randomized TM can be modeled as a deterministic TM with an extra tape of random bits, or as a machine with probabilistic transitions. Acceptance probability is taken over random choices.

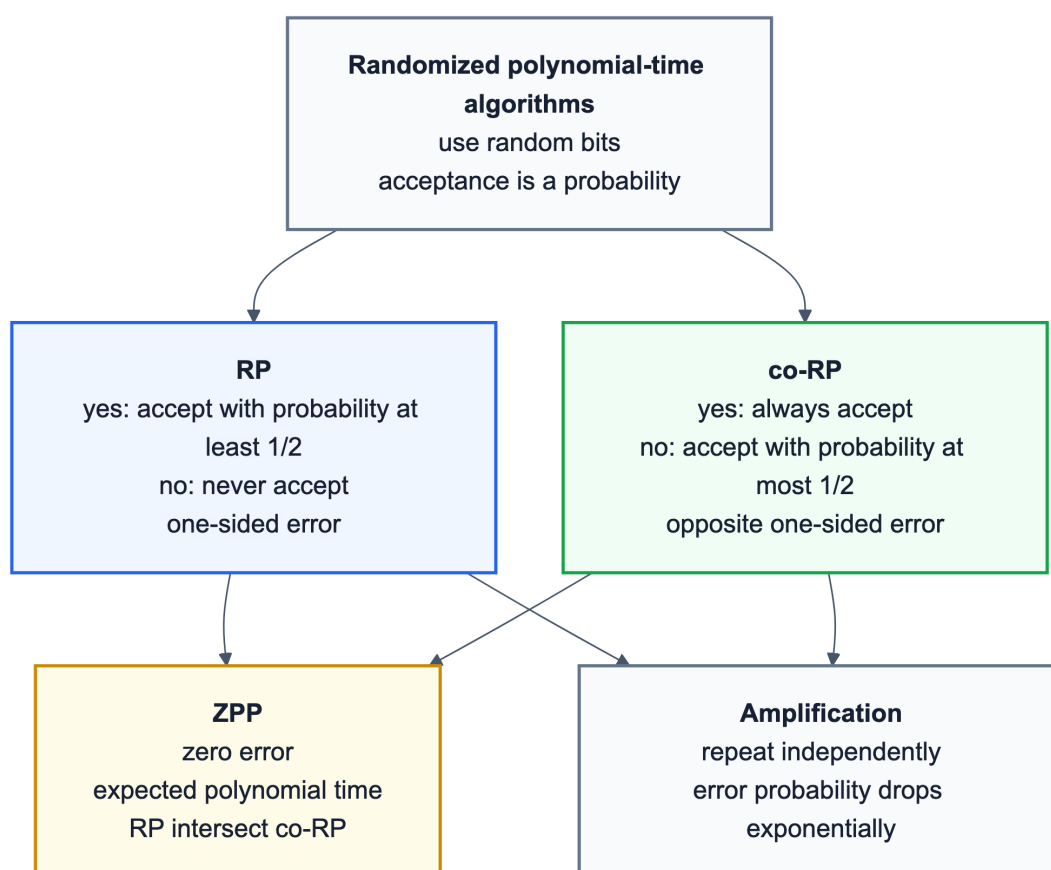


Figure 8: Randomized classes: RP and co-RP are one-sided-error classes, and ZPP is their zero-error intersection.

### RP

A language  $L$  is in RP if there is a polynomial-time randomized algorithm  $A$  such that:

$$x \in L \Rightarrow \Pr[A(x) = 1] \geq \frac{1}{2},$$

$$x \notin L \Rightarrow \Pr[A(x) = 1] = 0.$$

RP has one-sided error: it never falsely accepts no-instances, but may falsely reject yes-instances. Repeating independently reduces error exponentially.

### **co-RP**

$L \in \mathbf{coRP}$  iff  $\bar{L} \in \mathbf{RP}$ . Equivalently, there is a polynomial-time randomized algorithm with:

$$x \in L \Rightarrow \Pr[A(x) = 1] = 1,$$

$$x \notin L \Rightarrow \Pr[A(x) = 1] \leq \frac{1}{2}.$$

It never falsely rejects yes-instances, but may falsely accept no-instances.

### **ZPP**

ZPP is zero-error expected polynomial time. Equivalent characterizations:

$$\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP}.$$

Another view: algorithm outputs correct answer or “I do not know”; probability of “I do not know” is bounded away from 1, and repetition gives expected polynomial time.

Basic relationships:

$$\mathbf{P} \subseteq \mathbf{ZPP} \subseteq \mathbf{RP} \subseteq \mathbf{NP},$$

and

$$\mathbf{P} \subseteq \mathbf{ZPP} \subseteq \mathbf{coRP} \subseteq \mathbf{coNP}.$$

### **Related class: BPP**

BPP is not explicitly required in the exam outline, but it appears in one of the source notes and is useful context. It is the class of languages decidable by a polynomial-time randomized algorithm with bounded two-sided error:

$$\Pr[A(x) = \chi_L(x)] \geq \frac{2}{3}.$$

The constant  $2/3$  is arbitrary; any constant greater than  $1/2$  can be amplified by independent repetition and majority vote. RP and co-RP are one-sided-error subclasses of BPP:

$$\mathbf{RP} \subseteq \mathbf{BPP}, \quad \mathbf{coRP} \subseteq \mathbf{BPP}.$$

## 6. Decidability and Undecidability

### Recursive and recursively enumerable languages

A language  $L$  is **recursive** or **decidable** if some TM halts on every input and accepts exactly the strings in  $L$ .

A language  $L$  is **recursively enumerable (RE)** or **Turing-recognizable** if some TM accepts exactly strings in  $L$ ; on strings outside  $L$  it may reject or loop forever.

Relationships:

- Every decidable language is RE.
- $L$  is decidable iff both  $L$  and  $\overline{L}$  are RE.
- Some RE languages are undecidable.
- Some languages are not RE.

#### Simple explanation

Decidable means the machine always eventually answers yes or no. Recognizable means the machine eventually says yes for true cases, but for false cases it may run forever. That one-sided behavior is the whole source of many undecidability results.

### Universal language and universal Turing machine

Encode a TM  $M$  and input  $w$  as a string  $\langle M, w \rangle$ .

The universal language is

$$L_U = \{\langle M, w \rangle \mid M \text{ accepts } w\}.$$

A universal Turing machine simulates  $M$  on  $w$ . Thus  $L_U$  is RE: simulate  $M(w)$  and accept if it accepts.

However,  $L_U$  is not decidable. If it were decidable, the halting/acceptance problem could be solved, contradicting diagonalization.

### Diagonal language

Let  $M_1, M_2, \dots$  be an enumeration of Turing machines. Define the diagonal language

$$L_D = \{w_i \mid M_i \text{ does not accept } w_i\},$$

where  $w_i$  is the  $i$ -th string in a fixed enumeration.

If some TM decided  $L_D$ , then considering its own index yields contradiction: it would accept its own encoding iff it does not accept it.

This diagonal argument proves that there are undecidable languages and that some languages are not recursively enumerable.



## Halting problem

The halting problem asks:

$$HALT = \{\langle M, w \rangle \mid M \text{ halts on } w\}.$$

It is undecidable. Classic proof by contradiction:

1. Suppose decider  $H(M, w)$  says whether  $M$  halts on  $w$ .
2. Build machine  $D(x)$ : if  $H(x, x)$  says “halts”, loop forever; otherwise halt.
3. Run  $D(D)$ . If it halts, it must loop; if it loops, it must halt. Contradiction.

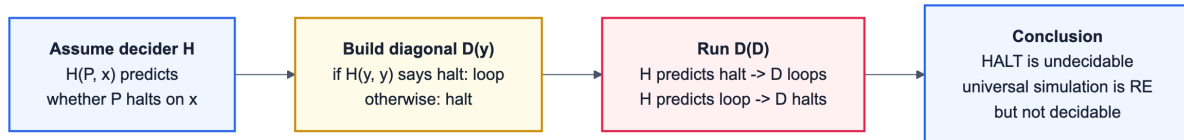


Figure 9: Diagonalization for the halting problem: assuming a decider lets us build a program that contradicts its own prediction.

### Example

Think of a hypothetical program `halts(program, input)`. The diagonal trick builds a new program that does the opposite of what `halts` predicts on itself. If `halts` says the new program halts, it loops; if `halts` says it loops, it halts. So `halts` cannot exist.

## Acceptance problem

The acceptance problem

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$$

is RE but undecidable. It is RE because a universal TM can simulate and accept when  $M$  accepts. It is undecidable by reduction from the halting problem or by direct diagonalization.

## Proving undecidability by reduction

To prove problem  $B$  undecidable:

1. Start from known undecidable problem  $A$ .
2. Construct a computable mapping from instances of  $A$  to instances of  $B$ .
3. Show that solving  $B$  would solve  $A$ .
4. Since  $A$  is undecidable,  $B$  must be undecidable.

Common source problems:  $HALT$ ,  $A_{TM}$ , emptiness/equivalence variants for TMs, Post correspondence problem.

## Decidable vs. semi-decidable behavior

For an RE but undecidable language, there is an algorithm that eventually confirms yes-instances, but no algorithm that always answers both yes and no.

For a co-RE but not decidable language, no-instances can be confirmed.

If neither  $L$  nor  $\bar{L}$  is RE, there is no recognizer for either side.

## Exam Checklist

Be able to:

- State and use  $O, \Omega, \Theta, o, \omega$ .
- Prove total correctness via invariant plus variant.
- Define DTM, multitape TM, and NTM, and explain why multitape TMs do not change polynomial classes.
- Translate decision problems to languages.
- Define P, NP, co-NP, NP-hard, NP-complete, and polynomial reduction.
- State Cook-Levin theorem and its computation-tableau idea.
- Explain exact, heuristic, and approximation approaches for NP-hard problems.
- Define PSPACE and NPSPACE and state Savitch theorem.
- Define RP, co-RP, and ZPP, including one-sided vs. zero error.
- Distinguish decidable, RE, and non-RE languages.
- Explain diagonalization, universal language, universal TM, and the halting problem.